

Fluxo de desenvolvimento iterativo

Acessar o contêiner LXC:

```
# Listar contêineres
lxc list

# Acessar o shell do contêiner de desenvolvimento
lxc exec maas-dev -- bash
```

Tornar-se o usuário de desenvolvimento:

```
# Dentro do contêiner, tornar-se o usuário padrão
su - ubuntu
```

Acessar o diretório de trabalho:

```
# Os arquivos do projeto estão em /work via bind mount
cd /work/maas
ls -la
```

Script de limpeza total (nuke):

Quando o MAAS fica em estado inconsistente e o `make snap-tree` não se recupera, executar o *script* de limpeza total:

```
#!/usr/bin/env bash
# nuke_maas.sh – Purge completo do MAAS e maas-test-db

if [ "$EUID" -ne 0 ]; then
    echo "❌ Error: This script must be run as root (sudo)."
```

```
    exit 1
fi

# Step 1: Parar serviços
systemctl stop snap.maas.pebble.service 2>/dev/null
systemctl disable snap.maas.pebble.service 2>/dev/null
systemctl stop snap.maas-test-db.*.service 2>/dev/null
systemctl disable snap.maas-test-db.*.service 2>/dev/null

# Step 2: Matar processos zumbis
killall -9 pebble postgres maas 2>/dev/null
sleep 2

# Step 3: Remover snaps com purge
snap remove --purge maas
snap remove --purge maas-test-db

# Step 4: Limpar diretórios residuais
rm -rf /var/snap/maas/
rm -rf /var/snap/maas-test-db/
rm -rf /home/ubuntu/snap/maas/
rm -rf /home/ubuntu/snap/maas-test-db/
rm -rf /root/snap/maas/
rm -rf /root/snap/maas-test-db/
rm -rf /var/lib/snapd/snap/maas/
rm -rf /var/lib/snapd/snap/maas-test-db/

# Step 5: Resetar systemd
systemctl daemon-reload
```

```
systemctl reset-failed
```

```
echo "Limpeza concluída."
```

Salvar como `nuke_maas.sh`, dar permissão de execução e rodar:

```
chmod +x nuke_maas.sh
```

```
sudo ./nuke_maas.sh
```

Após o *nuke*, recompilar o projeto com `make snap-tree`.

Restaurar banco de dados e configurar o MAAS:

Após recompilar, restaurar o banco de dados com dados de amostra:

```
#!/usr/bin/env bash
```

```
set -e
```

```
DB_SOCKET_DIR="/var/snap/maas-test-db/common/postgres/sockets"
```

```
DUMP_FILE="maasdb.dump"
```

```
SNAP_DUMP_PATH="/var/snap/maas-test-db/common/$DUMP_FILE"
```

```
DB_NAME="maassampledatab"
```

```
MAAS_URL="http://localhost:5240/MAAS/"
```

```
ADMIN_USER="admin"
```

```
ADMIN_PASS="adminpassword123"
```

```
ADMIN_EMAIL="admin@example.com"
```

```
echo "=== 1. Stopping MAAS snap services ==="
```

```
sudo snap stop maas
```

```
echo "=== 2. Cleaning up the database namespace ==="
```

```
sudo snap run --shell maas-test-db.psql -c "psql -U postgres -h $DB_SOCKET_DIR -d  
postgres -c 'DROP DATABASE IF EXISTS $DB_NAME;'"
```

```
echo "=== 3. Restoring the sample data dump ==="
```

```
if [ ! -f "$DUMP_FILE" ]; then
```

```
    echo "ERROR: $DUMP_FILE not found! Run 'make dumpdb DB_DUMP=$DUMP_FILE' first."
```

```
    exit 1
```

```
fi
```

```
sudo cp "$DUMP_FILE" "$SNAP_DUMP_PATH"
```

```
sudo snap run --shell maas-test-db.psql -c "db-dump restore \${SNAP_COMMON}/$DUMP_FILE  
$DB_NAME"
```

```
echo "=== 4. Granting privileges ==="
```

```
sudo snap run --shell maas-test-db.psql -c "psql -U postgres -h $DB_SOCKET_DIR -d  
$DB_NAME -c 'ALTER SCHEMA public OWNER TO maas; GRANT ALL ON SCHEMA public TO maas;'"
```

```
echo "=== 5. Running migrations ==="
```

```
sudo snap run --shell maas.pebble -c "maas-region dbupgrade"
```

```
echo "=== 6. Creating admin ==="
```

```
sudo snap run --shell maas.pebble -c "maas-region createadmin --username=$ADMIN_USER  
--password=$ADMIN_PASS --email=$ADMIN_EMAIL"
```

```
echo "=== 7. Starting MAAS ==="
```

```
sudo snap start maas
```

```
echo "=== 8. Waiting for API ==="
```

```
until curl -s -o /dev/null "$MAAS_URL"; do
```

```

        printf '.'
        sleep 2
done

echo "=== 9. Authenticating ==="
API_KEY=$(sudo snap run --shell maas.pebble -c "maas-region apikey --
user=$ADMIN_USER")
maas login "$ADMIN_USER" "$MAAS_URL" "$API_KEY"

echo "=== 10. Verifying machines ==="
maas "$ADMIN_USER" machines read | jq -r '.[[] |
[.hostname, .system_id, .power_type, .status_name] | @tsv'

```

Salvar como `update_maas.sh` e executar:

```

chmod +x update_maas.sh
./update_maas.sh

```

Script de deploy automatizado:

```

#!/bin/bash
# deploy-multiboot.sh – Automated multi-boot deployment workflow
#
# Usage:
#   ./deploy-multiboot.sh                # uses /home/ubuntu/layout.yaml
#   ./deploy-multiboot.sh /path/to/layout.yaml
#
# Prerequisites:
#   - maas CLI logged in: maas login admin <url> <api-key>
#   - Profile name "admin" (override with MAAS_PROFILE env var)

set -euo pipefail

PROFILE="${MAAS_PROFILE:-admin}"
LAYOUT_FILE="${1:-/home/ubuntu/layout.yaml}"

# Step 0: Verify prereqs
if ! maas "$PROFILE" machines read >/dev/null 2>&1; then
    echo "❌ Not logged into MAAS. Run: maas login $PROFILE <url> <api-key>"
    exit 1
fi

if [[ ! -f "$LAYOUT_FILE" ]]; then
    echo "❌ Layout file not found: $LAYOUT_FILE"
    exit 1
fi

# Step 1: List commissioned (Ready) machines
MACHINES_JSON=$(maas "$PROFILE" machines read 2>/dev/null)

MACHINES=$(echo "$MACHINES_JSON" | python3 -c "
import sys, json
data = json.load(sys.stdin)
ready = [m for m in data if m.get('status_name') == 'Ready']
if not ready:
    print('NONE')
    sys.exit(0)
for i, m in enumerate(ready):

```

```

        hostname = m.get('hostname', 'unknown')
        sys_id = m.get('system_id', '???')
        arch = m.get('architecture', '')
        print(f'{i+1}. {hostname:<20} {sys_id:<10} {arch}')
    ")

if [[ "$MACHINES" == "NONE" ]]; then
    echo "❌ No commissioned (Ready) machines found."
    exit 1
fi

echo "$MACHINES"

# Step 2: User selection
COUNT=$(echo "$MACHINES" | wc -l | tr -d ' ')
read -r -p "Select machine [1-$COUNT]: " SELECTION

SYSTEM_ID=$(echo "$MACHINES_JSON" | python3 -c "
import sys, json
data = json.load(sys.stdin)
ready = [m for m in data if m.get('status_name') == 'Ready']
try:
    idx = int($SELECTION) - 1
    print(ready[idx]['system_id'])
except (IndexError, ValueError):
    print('ERROR', file=sys.stderr)
    sys.exit(1)
")

echo "✅ Selected: $SYSTEM_ID"

# Step 3: Remove existing layout
maas "$PROFILE" machine delete-layout "$SYSTEM_ID" >/dev/null 2>&1 || true

# Step 4: Upload layout
maas "$PROFILE" machine set-layout "$SYSTEM_ID" layout@="$LAYOUT_FILE" >/dev/null
echo "✅ Layout uploaded."

# Step 5: Verify layout
maas "$PROFILE" machine get-layout "$SYSTEM_ID"

# Step 6: Deploy
maas "$PROFILE" machine multi-boot-deploy "$SYSTEM_ID"
echo "✅ Deploy triggered for $SYSTEM_ID"

```

Salvar como deploy-multiboot.sh:

```

chmod +x deploy-multiboot.sh
./deploy-multiboot.sh

```

Quando o make snap-tree falha (disco cheio, dependências quebradas), executar:

```

#!/bin/bash
set -euo pipefail

echo "[fix] Limpando diretórios temporários..."
sudo rm -rf /tmp/go-build*

```

```
sudo rm -rf /work/maas/dev-snap

echo "[fix] Estendendo volume LVM se necessário..."
sudo lvextend -l +100%FREE /dev/mapper/ubuntu--vg-ubuntu--lv 2>/dev/null || true
sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv 2>/dev/null || true

echo "[fix] Limpando cache do snap..."
sudo snap remove maas --purge 2>/dev/null || true

echo "[fix] Reconstruindo..."
cd /work/maas
make clean 2>/dev/null || true
make snap-tree
```

Salvar como fix-build.sh no diretório do projeto e executar:

```
chmod +x fix-build.sh
./fix-build.sh
```

Construir o projeto após iteração:

Após modificar o código fonte:

```
# 1. Sincronizar alterações do host para o contêiner
sudo make snap-tree-sync

# 2. Reiniciar os serviços do MAAS
sudo snap restart maas

# 3. Verificar se todos os serviços estão ativos
sudo maas status

# 4. Verificar logs em caso de erro
sudo snap logs maas -f
```